

Lab 7: CNN Text Classification

A small 1-D CNN, then parallel kernels, on SMS spam.

Work in a **Jupyter** notebook. Number the exercises. Do notes **7.1** through **7.3** first. You will need `pandas`, `numpy`, and `tensorflow` (Keras). A laptop CPU is enough if you keep the network small.

Download [spam.csv](#) into the same folder as the notebook. Open it with `encoding="latin1"`. `v2` is the message; `v1` is `ham` / `spam`.

When you are done: **File** → **Download as** → **HTML**, then upload the HTML on Canvas. Save the notebook before you export.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Embedding, Conv1D, GlobalMaxPooling1D, Dense, Dropout
```

If `tensorflow` is not installed: `pip install tensorflow`.

1. Load and encode (1 point)

1. Read `spam.csv` with `encoding="latin1"`. Keep `v1` and `v2`. Drop empty messages.
2. Map `ham` → `0` and `spam` → `1`. Print the class counts. Spam is the minority class.
3. Split **70% train / 30% test**, stratified on the label.

2. Tokens to a matrix (1 point)

1. Fit a Keras `Tokenizer` on the **training** messages only (`num_words` around 3000–5000 is enough).
2. Convert train and test text to integer sequences. Pad / truncate to a fixed `maxlen` (40–80 is plenty for SMS).

Print `vocab` size, `maxlen`, and the shape of the two padded matrices. A row should look like a list of integers, not a TF-IDF vector.

3. A small 1-D CNN (1 point)

Build a Keras model with this spine (you may change the widths):

1. `Embedding` (50–100 dimensions is enough; you may learn it from scratch).
2. At least one `Conv1D` (for example 64 filters, kernel size 3).
3. `GlobalMaxPooling1D`.
4. A `Dense` hidden layer with dropout if you want it.
5. A single sigmoid unit for binary spam/ham.

Print `model.summary()`. In a comment: what does the kernel of size 3 see, and what does global max keep?

4. Train and score (1 point)

Train for a few epochs (`batch_size` 32 or 64). Use validation data or a validation split. Predict on the test set (threshold 0.5 unless you choose another and say why).

Print accuracy, precision, recall, and F1 for the spam class, plus a classification report.

The class split is uneven. A model that always says `ham` will look accurate and still be useless.

5. Parallel kernels (1 point)

Add a second model that follows note **7.3: two or three** `Conv1D` branches with different kernel sizes (for example 2, 3, and 5), concatenate the pooled vectors, then classify.

You can use the Keras functional API for the branches. Train it the same way. Print the test F1 next to the single-kernel model.

In two or three sentences: did the extra kernel sizes help, and would you keep them on this SMS set?